

23110156_NguyenMinhHoang__ASP_Lab04

Link Github: https://github.com/tin1508/Asp.Net_TTH/tree/Lab04

Câu 1: Mini Stationery Store Catalog MVC

Kiến trúc Projects:

LABS_4	28
StationeryStore.Mvc	67
bin	74
Controllers	75
CategoriesController.cs	76
DataHealthController.cs	77
HomeController.cs	78
OrdersController.cs	79
StationeriesController.cs	80
Data	81
AppDbContext.cs	82
Exception	83
AppException.cs	84
ErrorCode.cs	85
GlobalExceptionHandler.cs	86
Migrations	87
20260614051527_InitialCreate.cs	88
20260614051527_InitialCreate.Designer.cs	89
20260614165744_AddCustomerNameToOrder.cs	90
20260614165744_AddCustomerNameToOrder.Designer.cs	91
AppDbContextModelSnapshot.cs	92
Models	93
Category.cs	94
ErrorViewModel.cs	95
OrderDetail.cs	96
Stationery.cs	97
StationeryOrder.cs	98
obj	99
Options	100
AppSettings.cs	101
StationeryStoreSettings.cs	102
Properties	103
Repositories	104
RepositoriesConfig	105
IRepository.cs	106
Repository.cs	107
CategoryRepository.cs	108
ICategoryRepository.cs	109
IStationeryOrderRepository.cs	110
IStationeryRepository.cs	111

Services	75
CategoryService.cs	76
DashboardService.cs	77
DataHealthService.cs	78
OrderStationeryService.cs	79
StationeryService.cs	80
ViewModels	81
CategoryCreateViewModel.cs	82
CategoryDetailViewModel.cs	83
CategoryListItemViewModel.cs	84
CategorySearchViewModel.cs	85
DashBoardViewModel.cs	86
DataHealthViewModel.cs	87
LowStockStationeryViewModel.cs	88
OrderCreateViewModel.cs	89
OrderDetailViewModel.cs	90
OrderItemViewModel.cs	91
OrderListItemViewModel.cs	92
StationeryCreateViewModel.cs	93
StationeryDetailViewModel.cs	94
StationeryFilterViewModel.cs	95
StationeryListItemViewModel.cs	96
StationerySearchViewModel.cs	97
StationerySelectViewModel.cs	98
StationeryStatsViewModel.cs	99
Views	100
Categories	101
Create.cshtml	102
Detail.cshtml	103
Index.cshtml	104
Search.cshtml	105
DataHealth	106
Index.cshtml	107
Home	108
Index.cshtml	109
Privacy.cshtml	110

LABS_4	28
StationeryStore.Mvc	67
Views	74
DataHealth	75
Home	76
Index.cshtml	77
Privacy.cshtml	78
Orders	79
Create.cshtml	80
Index.cshtml	81
Shared	82
_Layout.cshtml	83
_Layout.cshtml.css	84
StationeryCard.cshtml	85
_ValidationScriptsPartial.cshtml	86
Error.cshtml	87
Stationeries	88
Create.cshtml	89
Detail.cshtml	90
Filter.cshtml	91
Index.cshtml	92
Search.cshtml	93
Stats.cshtml	94
_ViewImports.cshtml	95
_ViewStart.cshtml	96
wwwroot	97
css	98
site.css	99
js	100
lib	101
favicon.ico	102
appsettings.Development.json	103
appsettings.json	104
Program.cs	105
StationeryStore.Mvc.csproj	106
~\$Lab_04.docx	107
~\$WRL0003.tmp	108
Lab_04.docx	109
StationeryStore.Mvc.slnx	110

File Program.cs:

```
// Add services to the container.
builder.Services.AddControllersWithViews();

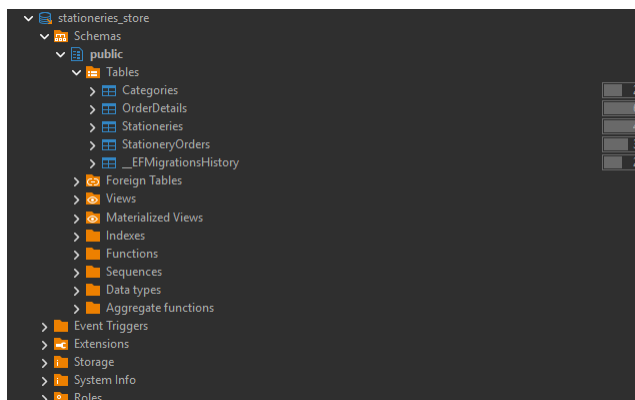
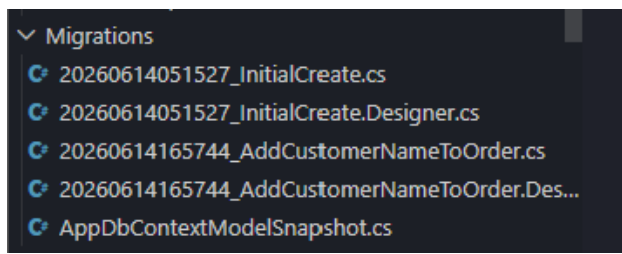
//add global exception handler
builder.Services.AddExceptionHandler<GlobalExceptionHandler>();

//add appsetting configuration
builder.Services.Configure<AppSettings>(builder.Configuration.GetSection("AppSettings"));
builder.Services.Configure<StationeryStoreSettings>(builder.Configuration.GetSection("StationeryStoreSettings"));

//register database
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseNpgsql(connectionString));

//register repositories
builder.Services.AddScoped<IStationeryRepository, StationeryRepository>();
builder.Services.AddScoped<ICategoryRepository, CategoryRepository>();
builder.Services.AddScoped<IStationeryOrderRepository, StationeryOrderRepository>();
//register services
builder.Services.AddScoped<IStationeryService, StationeryService>();
builder.Services.AddScoped<ICategoryService, CategoryService>();
builder.Services.AddScoped<IOrderStationeryService, OrderStationeryService>();
builder.Services.AddScoped<IDataHealthService, DataHealthService>();
builder.Services.AddScoped<IDashboardService, DashboardService>();
```

Database và Migrations:



EF core & Transaction:

AsNoTracking:

Em sử dụng AsNoTracking còn ở nhiều repository khác nữa và sẽ vào các phương thức là đọc dữ liệu. Dưới đây là 2 phương thức của StationeryRepository dùng AsNoTracking:

```
2 references
public async Task<List<Stationery>> searchStationeryByKeywordAndMinPrice(string? keyword, decimal? minPrice)
{
    var stationeries = await _context.Stationeries.Where(s => EF.Functions.ILike(s.Name, $"%{keyword}%") && s.Price >= minPrice).AsNoTracking().ToListAsync();
    return stationeries;
}
```

```

public async Task<IEnumerable<Stationery>> GetFilteredAsync(int? categoryId, decimal? minPrice, decimal? maxPrice)
{
    var query = _context.Stationeries
        .Include(s => s.Category)
        .AsNoTracking()
        .AsQueryable();

    if (categoryId.HasValue && categoryId.Value > 0)
    {
        query = query.Where(s => s.CategoryId == categoryId.Value);
    }

    if (minPrice.HasValue)
    {
        query = query.Where(s => s.Price >= minPrice.Value);
    }

    if (maxPrice.HasValue)
    {
        query = query.Where(s => s.Price <= maxPrice.Value);
    }

    return await query.ToListAsync();
}

```

Transaction khi tạo đơn hàng:

```

2 references
public async Task<StationeryOrder> CreateNewOrder(OrderCreateViewModel model)
{
    var order = new StationeryOrder
    {
        CustomerName = model.CustomerName.Trim(),
        CreatedAt = DateTime.UtcNow
    };

    foreach (var item in model.Items.Where(i => i.StationeryId > 0 && i.Quantity > 0))
    {
        var stationery = await _stationeryRepository.GetByIdAsync(item.StationeryId);
        if (stationery == null) throw new ApplicationException(ErrorCode.NOT_EXISTED_STATIONERY);
        if (stationery.Quantity < item.Quantity)
        {
            throw new ApplicationException(ErrorCode.NOT_ENOUGH_QUANTITY);
        }

        stationery.Quantity -= item.Quantity;

        order.OrderStationeries.Add(new OrderDetail
        {
            StationeryId = item.StationeryId,
            Quantity = item.Quantity,
            UnitPrice = stationery.Price,
        });
    }

    if (!order.OrderStationeries.Any())
    {
        throw new ApplicationException(ErrorCode.AT_LEAST_STATIONERY_WHEN_ORDERING);
    }

    order.TotalAmount = order.OrderStationeries.Sum(od => od.SubTotal);

    await _stationeryOrderRepository.AddAsync(order);
    await _stationeryOrderRepository.SaveChangeAsync();
    return order;
}

```

Giao diện người dùng (UI):

Trang chủ (dashboard):

Ở trang này, em đã làm thêm được tính phần trăm văn phòng phẩm còn hàng trong kho. Thêm vào đó, trang chủ này hiển thị 1 cách tổng quát các thông số của cửa hàng như là: Tổng số sản phẩm, số lượng danh mục, đơn hàng,... Trong các bài lab sau em sẽ phát triển thêm để hoàn thiện nhất có thể.

StationeryStore.Mvc

[Home](#)[Stationeries](#)[Categories](#)[Orders](#)[Data Health](#)[Privacy](#)

Stationery Store Dashboard

Quản lý tồn kho, danh mục và đơn hàng văn phòng phẩm.

16

TỔNG SẢN PHẨM

4

DANH MỤC

3

ĐƠN HÀNG

5

SẮP HẾT HÀNG

3

HẾT HÀNG

50%

TỔNG HÀNG

DANH MỤC	SỐ SẢN PHẨM
Bút viết	4 sản phẩm
Giấy & Vở	4 sản phẩm
Dụng cụ vẽ	4 sản phẩm
Văn phòng phẩm	4 sản phẩm

Quản lý sản phẩm

Quản lý danh mục

Xem đơn hàng

Sắp hết hàng (5)

Bút mực xanh Pilot

Còn 3

Tập học sinh 200 trang

Còn 4

Compa vẽ đường tròn

Còn 2

Bảng kẻ trong 2cm

Còn 1

Hũ dán nước 100ml

Còn 3

Hết hàng (3)

Bút da quang vàng

Hết hàng

Giấy A4 Double A 500 tờ

Hết hàng

Bộ chỉ màu 24 cây Faber-Castell

Hết hàng

Trang quản lý sản phẩm:

StationeryStore.Mvc

[Home](#)[Stationeries](#)[Categories](#)[Orders](#)[Data Health](#)[Privacy](#)

Stationeries Catalog

Tìm Kiếm

Tạo Mới

Xem thống kê tồn kho

Tổng số sản phẩm

16

Bút mực xanh Pilot

SKU: PEN-003

Danh mục:

12,000 VND

Số lượng: 3

Need to add more stock

Chi tiết

Bút da quang vàng

SKU: PEN-004

Danh mục:

16,000 VND

Số lượng: 0

Out of stock

Chi tiết

Tập học sinh 200 trang

SKU: PAP-002

Danh mục:

25,000 VND

Số lượng: 4

Need to add more stock

Chi tiết

Giấy A4 Double A 500 tờ

SKU: PAP-003

Danh mục:

95,000 VND

Số lượng: 0

Out of stock

Chi tiết

Sổ tay bìa cứng A5

SKU: PAP-004

Danh mục:

35,000 VND

Số lượng: 50

In stock

Chi tiết

Thước kẻ 30cm

SKU: ART-001

Danh mục:

10,000 VND

Số lượng: 60

In stock

Chi tiết

Compa vẽ đường tròn

SKU: ART-002

Danh mục:

22,000 VND

Số lượng: 2

Need to add more stock

Chi tiết

Màu nước 12 màu Sakura

SKU: ART-003

Danh mục:

45,000 VND

Số lượng: 30

In stock

Chi tiết

Bộ chỉ màu 24 cây Faber-Castell

SKU: ART-004

Danh mục:

85,000 VND

Số lượng: 0

Out of stock

Chi tiết

Kéo văn phòng Hồng Hà

SKU: OFF-001

Danh mục:

20,000 VND

Số lượng: 40

In stock

Chi tiết

Bảng kẻ trong 2cm

SKU: OFF-002

Danh mục:

12,000 VND

Số lượng: 1

Need to add more stock

Chi tiết

Hũ dán nước 100ml

SKU: OFF-004

Danh mục:

10,000 VND

Số lượng: 3

Need to add more stock

Chi tiết

Trang tìm kiếm sản phẩm và tạo mới sản phẩm:

StationeryStore.Mvc

[Home](#)[Stationeries](#)[Categories](#)[Orders](#)[Data Health](#)[Privacy](#)

Search Stationeries

Form GET dùng Query String để tìm kiếm sản phẩm theo từ khóa và giá tối thiểu

Từ khóa

Ví dụ: sách, bút...

Giá tối thiểu

Ví dụ: 300000

Tìm Kiếm

Xóa

Search Result

Keyword:

Min Price:

không có sản phẩm nào trong hệ thống

Quay lại danh sách

© 2026 - StationeryStore.Mvc.Bộ hàngDev - Privacy

StationeryStore.Mvc

[Home](#)[Stationeries](#)[Categories](#)[Orders](#)[Data Health](#)[Privacy](#)

Tạo Văn Phòng Phẩm

Form POST dùng asp-for để bind dữ liệu vào StationeryCreateViewModel.

Quay lại danh sách

Mã sản phẩm

VI DỤ: NBK-AA-LND

Tên sản phẩm

VI DỤ: Blue BallPoint Pen

Danh mục

VI DỤ: Accessories

Nhà cung cấp

VI DỤ: EraserTech

Giá bán

0.00

Số lượng

1

Mức tồn tối thiểu

1

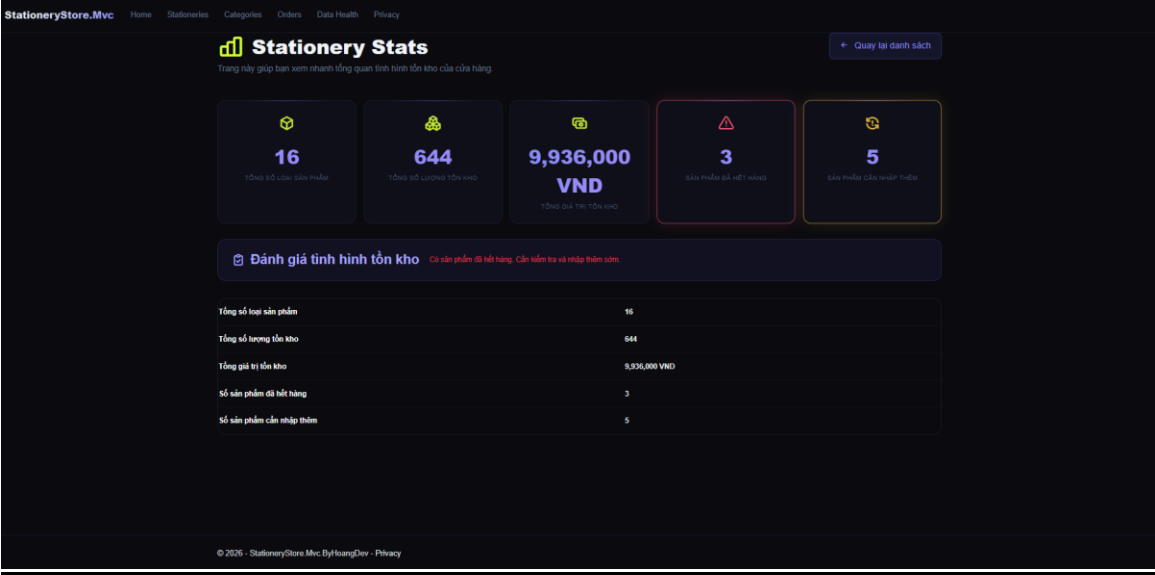
Lưu sản phẩm

Quay lại

Model Binding từ Form POST

Khi bấm Lưu, ASP.NET Core sẽ đọc các input trong form và tự map vào StationeryCreateViewModel.
Ví dụ input có `asp-for="Name"` thì dữ liệu sẽ được bind vào property `Name`.
Nếu tên input không khớp với property, action có thể nhận null hoặc giá trị mặc định.

Trang xem thống kê riêng đôi với văn phòng phẩm:



Trang quản lý danh mục:

StationeryStore.Mvc

[Home](#)[Stationeries](#)[Categories](#)[Orders](#)[Data Health](#)[Privacy](#)

Category Catalog

Tìm Kiếm

Tạo Mới

Đây là trang danh sách danh mục sản phẩm của cửa hàng.

Tổng số danh mục sản phẩm

4

TÊN MỤC	SỐ LƯỢNG SẢN PHẨM	CHI TIẾT
Đồ viết	4	Xem
Giấy & Vở	4	Xem
Dụng cụ vẽ	4	Xem
Văn phòng phẩm	4	Xem

← Quay lại trang chủ

© 2025 - StationeryStore.Mvc, ByHoangDev - Privacy

Trang tìm kiếm và tạo mới danh mục:

StationeryStore.Mvc

HomeStationeriesCategoriesOrdersData HealthPrivacy

Search Categories

Form GET dùng Query String để tìm kiếm danh mục sản phẩm theo từ khóa

Quay lại danh sách

Từ khóa

Ví dụ: Bút viết...

Tìm Kiếm

Xóa

Search Result

Keyword:

Tên danh mục sản phẩm

Chi tiết

© 2026 - StationeryStore.Mvc.ByluongDev - Privacy

StationeryStore.Mvc

HomeStationeriesCategoriesOrdersData HealthPrivacy

Tạo Danh Mục Văn Phòng Phẩm

Form POST dùng asp-for để bind dữ liệu vào CategoryCreateViewModel

Quay lại danh sách

Tên danh mục văn phòng phẩm

Ví dụ: Blue BallPoint Pen

Lưu

Quay lại

Model Binding từ Form POST

Khi bấm Lưu, ASP.NET Core sẽ đọc các input trong form và tự map vào CategoryCreateViewModel.

Ví dụ input có `asp-for="Name"` thì dữ liệu sẽ được bind vào property `Name`.

Nếu tên input không khớp với property, action có thể nhận null hoặc giá trị mặc định.

© 2026 - StationeryStore.Mvc.ByluongDev - Privacy

Trang quản lý đơn hàng:

StationeryStore.Mvc

HomeStationeriesCategoriesOrdersData HealthPrivacy

Order Catalog

Đây là trang danh sách danh mục sản phẩm của cửa hàng

Tạo Mới

Tổng số đơn hàng

3

MÃ ĐƠN HÀNG

TÊN KHÁCH HÀNG

THỜI GIAN TẠO ĐƠN

TỔNG TIỀN

292ba7a6-5552-4d89-ba24-421292b41c

Lu Lu

15/06/2026 18:13

180000.00

99818aa7-d9d7-4e79-8852-8992a8b3a0

Ủ Béo

15/06/2026 18:12

8000.00

ea2d475c-6d27-4336-ab8e-11d8ef8213e

Minh Hoàng

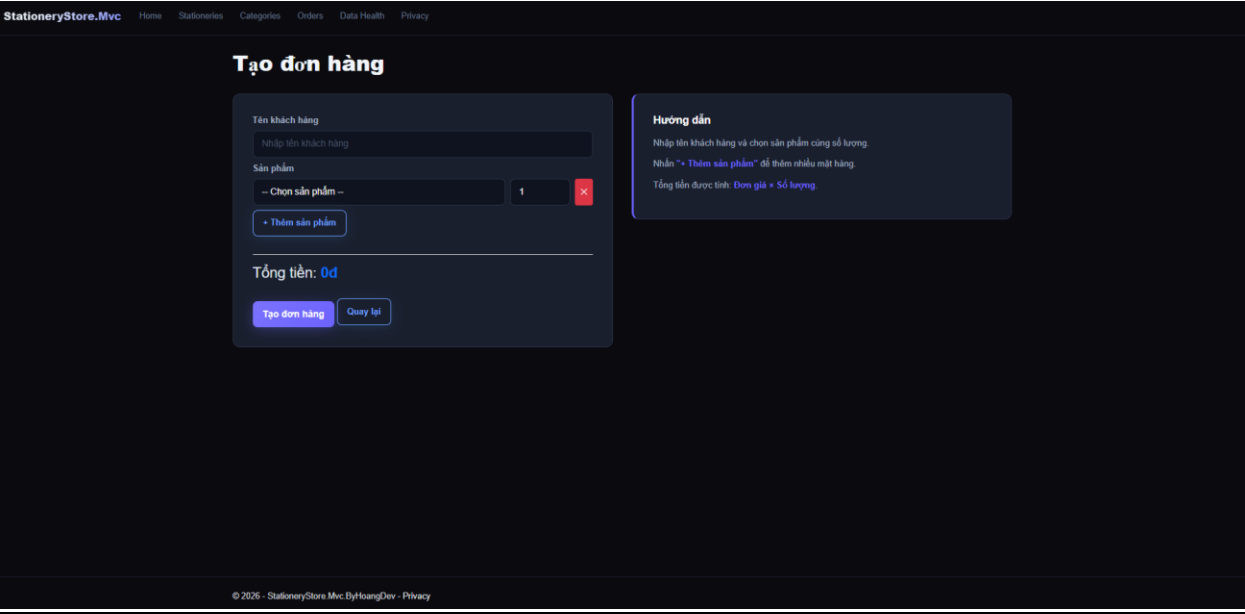
15/06/2026 18:02

48000.00

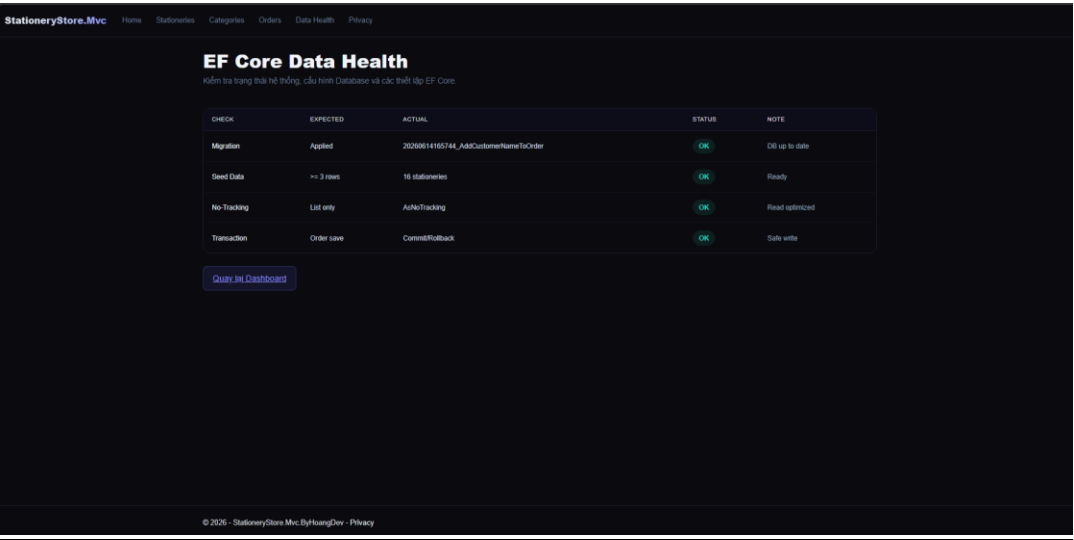
← Quay lại trang chủ

© 2026 - StationeryStore.Mvc.ByluongDev - Privacy

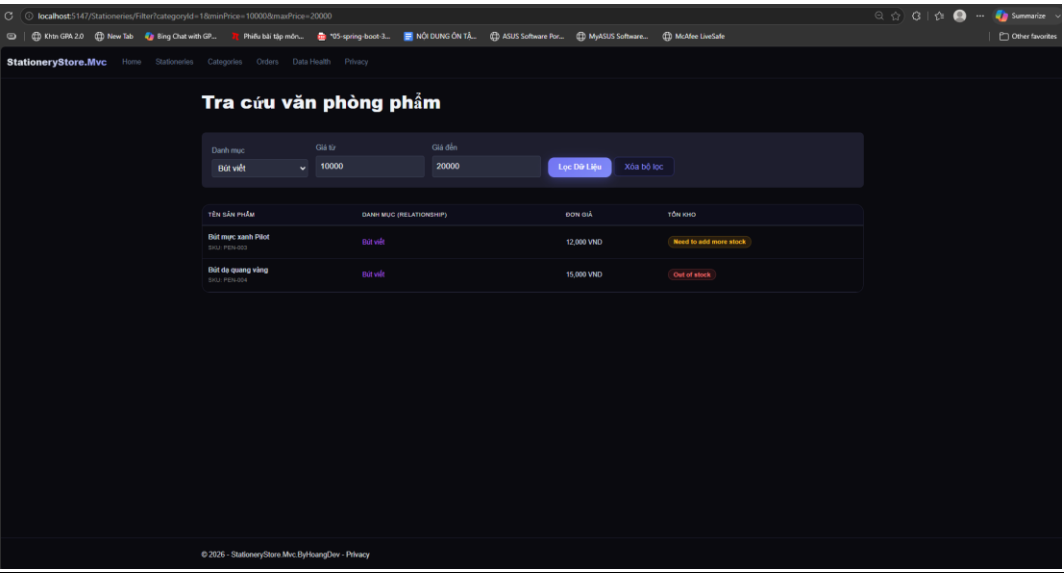
Trang tạo mới đơn hàng: (Ở đây ta có thể mua nhiều sản phẩm)



Trang Data Health:



Trang filter:



Câu 2:

- **Feature 1 — Cấu hình ngưỡng bằng Options Pattern**

Đầu tiên ta cấu hình ngưỡng tồn kho trong appsettings.json:

```
{
  "StationeryStoreSettings": {
    "LowStockThreshold": 10
  }
}
```

Tiếp đó, ta sẽ tạo StationeryStoreSettings trong StationeryStore.Mvc.Options với mục đích để ta có thể truy cập được thuộc tính của settings này.

```
namespace StationeryStore.Mvc.Options;

3 references
public class StationeryStoreSettings
{
    1 reference
    public int LowStockThreshold {get; set;}
}
```

Ta tiến hành dependency injection trong program.cs:

```
builder.Services.Configure<AppSettings>(builder.Configuration.GetSection("AppSettings"));
builder.Services.Configure<StationeryStoreSettings>(builder.Configuration.GetSection("StationeryStoreSettings"));
```

Tiến hành sử dụng threshold stock mà ta đã cấu hình trong dashboard service:

```
int threshold = _storeSettings.LowStockThreshold;

var lowStockItems = stationeries
    .Where(s => s.Quantity > 0 && s.Quantity <= threshold)
    .ToList();

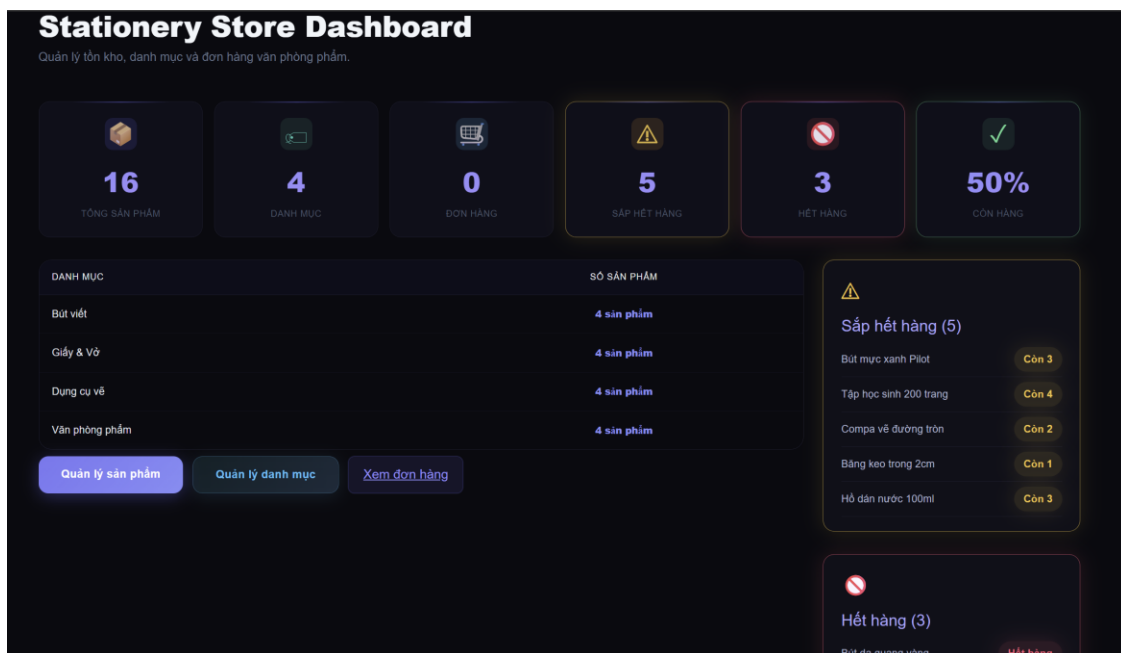
var outOfStockItems = stationeries
    .Where(s => s.Quantity == 0)
    .ToList();

int totalStationeries = _stationeries.Count();

int inStockPercent = totalStationeries == 0 ? 0 :
    (int)((double)stationeries.Count(s => s.Quantity > (s.MinStock > 0 ? s.MinStock : threshold)) / totalStationeries * 100);
```

Ta có thể kiểm tra threshold stock mà ta đã cấu hình hoạt động như thế nào như sau:

Với ngưỡng tồn kho lowstockThreshold là 10 thì trang dashboard của chúng ta sẽ hiển thị sản phẩm hết hàng là 5:

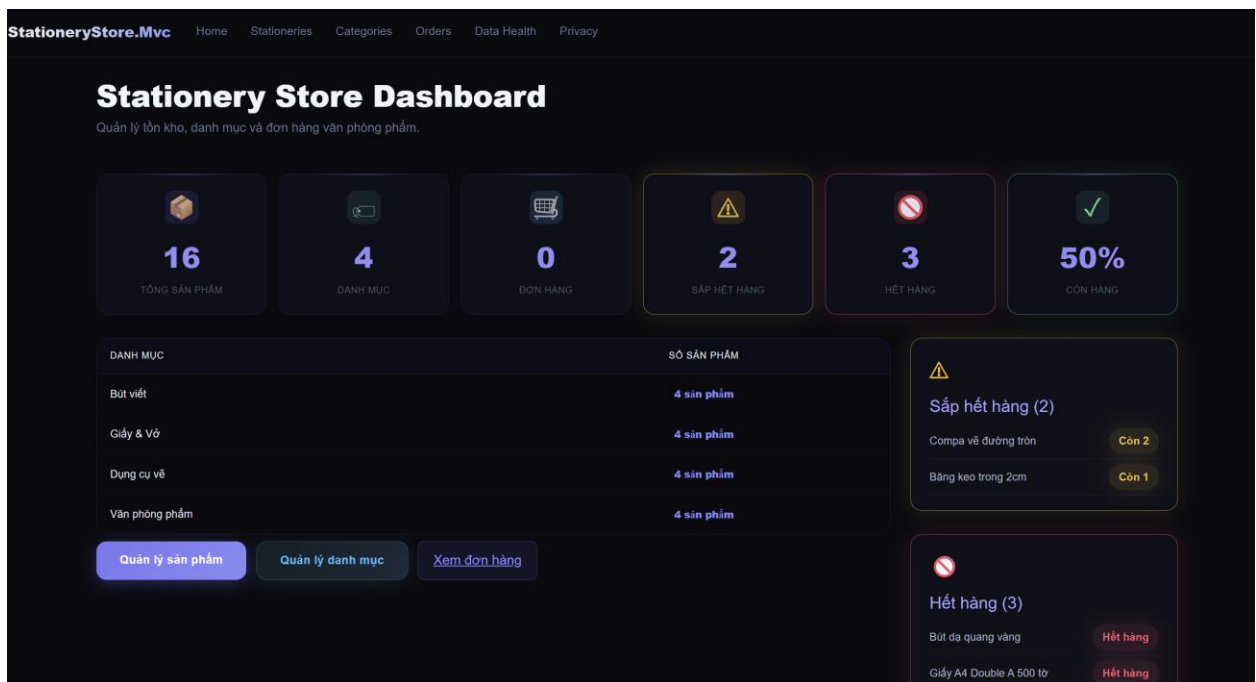


Ở trên ta có thể thấy sản phẩm sắp hết hàng là 5.

Nhưng nếu ta cấu hình lại LowStockThreshold là 2

```
},  
"StationeryStoreSettings": {  
  "LowStockThreshold": 2  
}
```

Và chạy lại dashboard để số lượng sản phẩm hết hàng:



Ta có thể thấy được số lượng sản phẩm sắp hết hàng đã giảm xuống còn 2.

Điều này cho chúng ta thấy được ta đã cấu hình và cài đặt thành công ngưỡng tồn kho mặc định của chương trình của chúng ta.

- **Feature 2 — Trang lọc dữ liệu theo Relationship và khoảng giá trị**

StationeryRepository:

```
2 references
public async Task<IEnumerable<Stationery>> GetFilteredAsync(int? categoryId, decimal? minPrice, decimal? maxPrice)
{
    var query = _context.Stationeries
        .Include(s => s.Category)
        .AsNoTracking()
        .AsQueryable();

    if (categoryId.HasValue && categoryId.Value > 0)
    {
        query = query.Where(s => s.CategoryId == categoryId.Value);
    }

    if (minPrice.HasValue)
    {
        query = query.Where(s => s.Price >= minPrice.Value);
    }

    if (maxPrice.HasValue)
    {
        query = query.Where(s => s.Price <= maxPrice.Value);
    }

    return await query.ToListAsync();
}
```

StationeryService:

```
2 references
public async Task<StationeryFilterViewModel> GetFilteredStationeriesAsync(int? categoryId, decimal? minPrice, decimal? maxPrice)
{
    var entities = await _stationeryRepository.GetFilteredAsync(categoryId, minPrice, maxPrice);

    var resultList = entities.Select(e => new StationeryListItemViewModel
    {
        Id = e.Id,
        Sku = e.Sku ?? string.Empty,
        Name = e.Name,
        Supplier = e.Supplier ?? string.Empty,
        Price = e.Price,
        Quantity = e.Quantity,
        MinStock = e.MinStock,
        Category = e.Category
    }).ToList();

    return new StationeryFilterViewModel
    {
        CategoryId = categoryId,
        MinPrice = minPrice,
        MaxPrice = maxPrice,
        Results = resultList
    };
}
```

```
namespace StationeryStore.Mvc.ViewModels;

5 references
public class StationeryFilterViewModel
{
    2 references
    public int? CategoryId { get; set; }
    2 references
    public decimal? MinPrice { get; set; }
    2 references
    public decimal? MaxPrice { get; set; }

    3 references
    public List<StationeryListItemViewModel> Results { get; set; } = new List<StationeryListItemViewModel>();
}
```

Ta sẽ lọc các văn phòng phẩm theo categoryId, minPrice, và maxPrice dựa trên Repository layer.

Tối ưu nhất có thể cho phần controller:

StationeryController:

```
[HttpGet]
0 references
public async Task<IActionResult> Filter(int? categoryId, decimal? minPrice, decimal? maxPrice)
{
    var model = await _stationeryService.GetFilteredStationeriesAsync(categoryId, minPrice, maxPrice);

    var categories = await _categoryService.GetAllCategoryListAsync();

    ViewBag.CategoryList = new SelectList(categories, "Id", "Name", categoryId);

    return View(model);
}
```

Khi ta gọi tới route:

<http://localhost:5147/Stationeries/Filter?categoryId=1&minPrice=10000&maxPrice=20000>

Chương trình sẽ lọc những văn phòng phẩm theo danh mục là Id = 1 và có giá từ khoảng 10,000 đồng đến 20,000 đồng.

The screenshot shows a web browser displaying the StationeryStore.Mvc application. The page title is "Tra cứu văn phòng phẩm". The navigation bar includes links for Home, Stationeries, Categories, Orders, Data Health, and Privacy. The main content area features a filter section with a dropdown menu for "Danh mục" (Category) set to "Bút viết", and input fields for "Giá từ" (Min Price) set to 10000 and "Giá đến" (Max Price) set to 20000. There are buttons for "Lọc Dữ Liệu" (Filter Data) and "Xóa bộ lọc" (Clear filter). Below the filter section is a table with the following data:

TÊN SẢN PHẨM	DANH MỤC (RELATIONSHIP)	ĐƠN GIÁ	TỒN KHO
Bút mực xanh Pilot SKU: PEN-003	Bút viết	12,000 VND	Need to add more stock
Bút dạ quang vàng SKU: PEN-004	Bút viết	15,000 VND	Out of stock

At the bottom of the page, there is a footer with the text: "© 2026 - StationeryStore.Mvc.ByHoangDev - Privacy".

- **Feature 3 — Thay đổi Entity và tạo Migration mới**

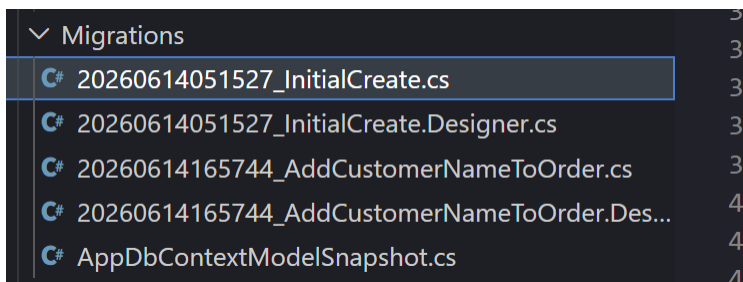
Ở đây, ban đầu order không có customer name:

```
});

migrationBuilder.CreateTable(
    name: "StationeryOrders",
    columns: table => new
    {
        Id = table.Column<string>(type: "text", nullable: false),
        CreatedAt = table.Column<DateTime>(type: "timestamp with time zone", nullable: false),
        TotalAmount = table.Column<decimal>(type: "numeric(10,2)", nullable: false)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_StationeryOrders", x => x.Id);
    }
});
```

Nhưng khi tạo form để tạo order thì cần thêm 1 thuộc tính customer name, nên ta đã thêm customername vào migrations lại:

Kết quả thì ta sẽ được như sau:



Bên cạnh đó, ta cũng có thể kiểm tra bằng DataHealth để biết tình trạng của migrations:

StationeryStore.Mvc Home Stationeries Categories Orders Data Health Privacy

EF Core Data Health

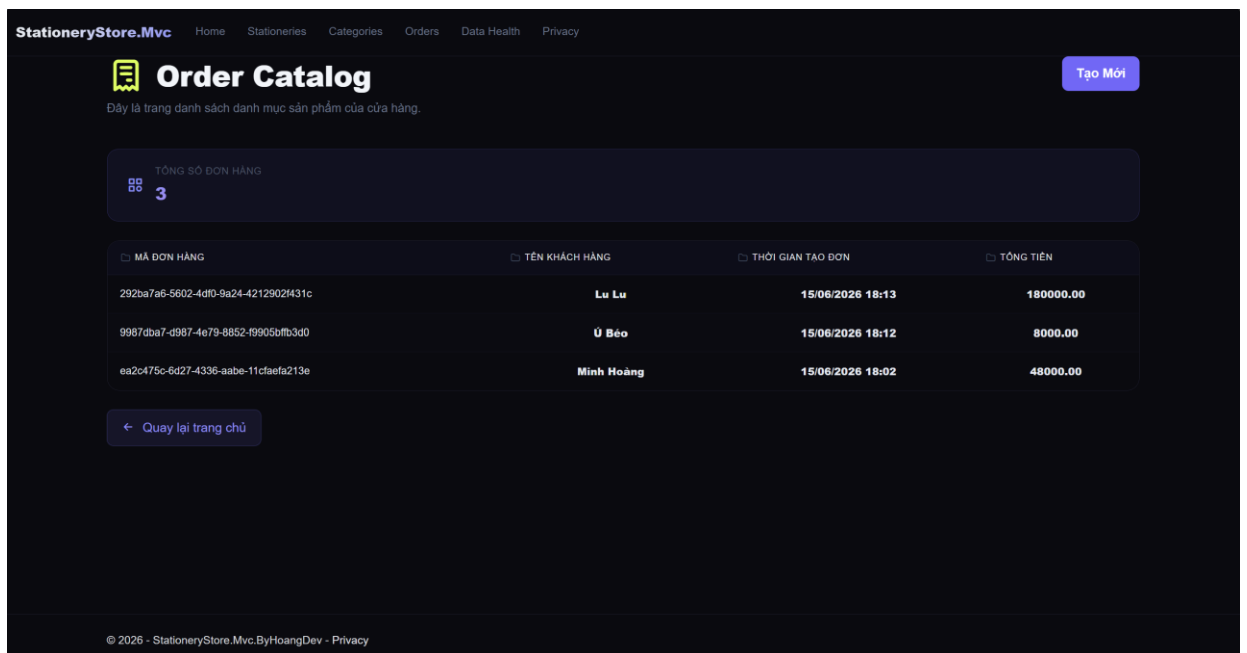
Kiểm tra trạng thái hệ thống, cấu hình Database và các thiết lập EF Core.

CHECK	EXPECTED	ACTUAL	STATUS	NOTE
Migration	Applied	20260614165744_AddCustomerNameToOrder	OK	DB up to date
Seed Data	>= 3 rows	16 stationeries	OK	Ready
No-Tracking	List only	AsNoTracking	OK	Read optimized
Transaction	Order save	Commit/Rollback	OK	Safe write

[Quay lại Dashboard](#)

© 2026 - StationeryStore.Mvc.ByHoangDev - Privacy

- **Feature khuyến khích — Trang lịch sử nghiệp vụ**



Ở đây, ta có thể thấy được lịch sử tạo đơn hàng của cửa hàng văn phòng phẩm. Lịch sử của đơn hàng được ghi lại vào thời điểm nào và được mua bởi ai.

Câu 3:

1. Nếu đăng ký ApplicationDbContext là Singleton trong một web app có nhiều request đồng thời, rủi ro gì có thể xảy ra? Em sẽ chọn lifetime nào và vì sao?

DbContext không an toàn khi chạy đa luồng. Nếu đăng ký Singleton, tất cả user truy cập web cùng lúc sẽ xài chung một instance ApplicationDbContext. Nếu User A đang mua "Bút bi" và User B đang mua "Vở", khi gọi SaveChangesAsync(), dữ liệu sẽ bị ghi đè, xung đột (Concurrency Exception) hoặc rò rỉ bộ nhớ do Change Tracker càng ngày càng to và không giảm.

Em sẽ lựa chọn Scoped vì mỗi HTTP Request sẽ được cấp một DbContext riêng biệt, sống và kết thúc cùng request đó. Điều này đảm bảo an toàn dữ liệu và tối ưu RAM.

2. Nếu Controller trực tiếp query DbContext và chứa luôn logic nghiệp vụ, việc bảo trì, tái sử dụng và kiểm tra code sẽ gặp khó khăn gì?

Bảo trì & Tái sử dụng: Nếu HomeController tự viết LINQ để đếm số lượng văn phòng phẩm sắp hết hàng (`_context.Stationeries.Where(...)`), khi cần viết một API cho Mobile App hiển thị con số này, ta phải copy-paste lại đoạn LINQ đó. Code bị lặp. Điều này sẽ rất khó mở rộng và bảo trì trong tương lai.

Kiểm thử (Testing): Không thể viết Unit Test cho logic này mà không kết nối với SQL Server thật, do Controller gắn chặt với ApplicationDbContext.

3. Trong feature ngưỡng tồn kho hoặc ngưỡng chỗ trống ở Câu 2, rule xác định trạng thái nên đặt ở Controller, Service hay Repository? Giải thích lựa chọn của em.

Đặt tại **Service Layer** (cụ thể là DashboardService ở Câu 1).

Vì Repository chỉ làm nhiệm vụ lấy Data (truy vấn DB) không chứa nghiệp vụ. Controller chỉ nhận Request và trả View. Việc tính toán xem một "Bút dạ quang" có thuộc diện sắp hết hàng hay không dựa vào Quantity, MinStock và cấu hình appsettings.json là **Business Logic (Nghiệp vụ)**. Service là để chứa logic này.

4. Nếu một Service tự new Repository bên trong method, tại sao việc thay Repository thật bằng Fake Repository để kiểm tra logic sẽ khó khăn?

Nếu StationeryService dùng `_stationeryRepository = new StationeryRepository()`, Service đã bị gắn cứng vào implementation thật.

Khi viết Unit Test, ta không muốn chạm vào Database thật. Nếu dùng Constructor Dependency Injection, ta có thể dễ dàng tiêm một FakeStationeryRepository (chỉ chứa list data giả trên RAM) vào để test. Việc tự new phá vỡ nguyên tắc **Dependency Inversion**, triệt tiêu khả năng tự động test logic.

5. Nếu em thêm một property mới vào Entity nhưng quên tạo migration, hoặc đã tạo migration nhưng quên chạy database update, ứng dụng có thể gặp lỗi gì?

Ứng dụng sẽ văng lỗi **SqlException: Invalid column name 'CustomerName'** khi thực thi bất kỳ truy vấn nào gọi đến bảng Categories vì code C# và cấu trúc thực tế của PostgreSQL bị lệch pha (Schema Mismatch). EF Core cố map dữ liệu vào cột CustomerName nhưng postgres không tìm thấy cột này.

6. Nếu trang danh sách đọc 10.000 record nhưng không dùng AsNoTracking, hệ thống có thể tổn thêm tài nguyên như thế nào? Khi nào không nên dùng AsNoTracking?

Khi không dùng AsNoTracking ở trang /Stationeries/Filter, EF Core sẽ phải đưa cả 10.000 object Stationery vào **Change Tracker** để theo dõi sự thay đổi. Điều này gây tốn gấp đôi dung lượng RAM và hao phí CPU vô ích cho việc quản lý trạng thái.

Trong nghiệp vụ **Tạo Đơn Hàng**. Ta cần EF Core theo dõi object Stationery để khi ta gọi `stationery.Quantity -= item.Quantity`, `SaveChangesAsync()` sẽ biết phát sinh lệnh UPDATE xuống DB. Nếu dùng AsNoTracking ở đây thì sẽ không gây ra bất kỳ sự thay đổi nào đối xuống với database. Ta chỉ nên dùng AsNoTracking() cho những action đọc dữ liệu và không có tương tác hay làm thay đổi dữ liệu.

7. Nếu nghiệp vụ tạo giao dịch đã lưu bản ghi chính nhưng bước trừ tồn kho hoặc giảm số chỗ bị lỗi, dữ liệu sẽ sai như thế nào nếu không có transaction?

Hệ thống sẽ lưu Order và OrderDetail thành công. Sau đó, trong quá trình cập nhật trừ Quantity của "Bút bi" thất bại (do cúp điện,...). Khách hàng đã được ghi nhận là mua thành công và phải trả tiền, nhưng trong kho số lượng Bút bi vẫn không bị trừ. Dữ liệu Kế toán và Kho hàng bị lệch hoàn toàn (Data Inconsistency).

8. Hãy mô tả cách em chủ động kiểm tra tình huống rollback. Em sẽ tạo dữ liệu đầu vào nào, kiểm tra những bảng nào và mong đợi kết quả gì?

Dữ liệu đầu vào: Tạo đơn hàng mua "Tập học sinh" số lượng 5. Can thiệp vào code, chủ động ném ra `throw new Exception("Test Rollback");` ngay sau dòng `_orderRepository.Add(order)` nhưng trước dòng `_context.SaveChangesAsync()`. Mở PostgreSQL, kiểm tra bảng Orders (không có đơn mới) và bảng Stationeries (Quantity Tập học sinh vẫn giữ nguyên). Giao dịch bị hủy hoàn toàn, đảm bảo tính nguyên vẹn dữ liệu.

9. Nếu config được đọc bằng string key ở nhiều nơi, khi đổi tên key hoặc đổi môi trường triển khai sẽ có rủi ro gì? Options Pattern giúp giảm rủi ro đó như thế nào?

Gọi `_config["StationeryStoreSettings:LowStockThresold"]` (gõ sai chữ Threshold). Trình biên dịch không báo lỗi, app chạy lên sẽ trả về null hoặc 0, gây sai lệch logic tồn kho một cách rất khó debug. Options Pattern giúp ánh xạ chuỗi JSON thành class `C# StationeryStoreSettings`. Nếu gõ sai tên biến, Visual Studio Code sẽ báo lỗi đỏ (Compile-time error) ngay lúc code.

10. Nếu cùng một request có Service và Repository đều cần DbContext, việc dùng Scoped lifetime giúp ích gì cho tính nhất quán của dữ liệu?

Trong 1 request, `OrderService` gọi `OrderRepository` (thêm đơn) và `StationeryRepository` (trừ tồn kho). Vì là Scoped, cả 2 Repository này đang được Inject **cùng một instance** của `AppDbContext`. Nhờ xài chung DbContext, cả 2 thao tác cùng chia sẻ chung một **Change Tracker**. Khi Service gọi `_context.SaveChangesAsync()` một lần duy nhất, cả thao tác thêm đơn và trừ kho đều được gom vào chung một Transaction của EF Core.

11. Seed Data có thể gây trùng dữ liệu trong trường hợp nào? Em sẽ thiết kế seed như thế nào để dễ kiểm soát và không tạo dữ liệu lặp ngoài ý muốn?

Seed data sẽ gây trùng dữ liệu (tạo ra hàng loạt data rác) nếu ta gọi lệnh `_context.Add(...)` trực tiếp trong lúc khởi chạy mà không có bước kiểm tra xem dữ liệu đó đã tồn tại trong Database hay chưa. Mỗi lần khởi động lại app, lệnh Insert lại bị gọi ngầm một lần.

Cách thiết kế để kiểm soát của em: Trong ứng dụng Stationery Store của em, em ưu tiên việc nhập liệu thực tế qua giao diện Form (CRUD). Để đảm bảo không có dữ liệu lặp ngoài ý muốn (cả khi người dùng nhập tay lẫn khi có kịch bản chạy Seed Data qua Service), em thiết lập logic **Check-then-Insert (Kiểm tra trước khi chèn)** ngay tại tầng Service.

Ví dụ:

Trong đoạn code này, bản chất em đã cài đặt id của stationery là kiểu generate tự động kiểu int. Nên ta không cần nhập id khi tạo. Bên cạnh đó, sku là mã hàng của mỗi mặt hàng, nên nó phải là duy nhất. Nhờ vào bản chất unique của sku nên ta luôn kiểm tra xem nếu có mặt hàng nào đó được tạo bị trùng mã hàng với mặt hàng đã tồn tại trong

database. Điều này giúp dữ liệu của chúng ta nhất quán và không bị trùng lặp.

```
public async Task<Stationery> CreateNewStationery(StationeryCreateViewModel model)
{
    var stationery = new Stationery();
    string sku = model.Sku;
    if(sku != null)
    {
        if(_stationeryRepository.existedStationery(sku))
        {
            throw new AppException(ErrorCode.EXISTED_STATIONERY);
        }
        stationery.Sku = sku;
    }
    var category = await _categoryRepository.GetCategoryByName(model.CategoryName.Trim());
    if(category == null) throw new AppException(ErrorCode.NOT_EXISTED_CATEGORY);
    stationery.Name = model.Name;
    stationery.CategoryId = category.Id;
    stationery.Supplier = model.Supplier;
    stationery.Price = model.UnitPrice;
    stationery.Quantity = model.Quantity;
    stationery.MinStock = model.MinStock;
    stationery.LastUpdatedAt = DateTime.UtcNow;
    await _stationeryRepository.AddAsync(stationery);
    Console.WriteLine($"UTC Now: {DateTime.UtcNow}");
    Console.WriteLine($"Local Now: {DateTime.Now}");
    Console.WriteLine($"Server Timezone: {TimeZoneInfo.Local.DisplayName}");
    await _stationeryRepository.SaveChangeAsync();
    return stationery;
}
```

 You have Docker installed on your system. Please install the recommended extensions for Docker.

12. Nếu cần mở rộng transaction từ một item thành nhiều item, em cần kiểm tra những điều kiện nào trước khi commit để tránh tạo giao dịch nửa đúng nửa sai?

Khi đơn hàng có nhiều món (Bút, Vở, Thuốc), trước khi Commit transaction, phải kiểm tra vòng lặp:

Tính hợp lệ: Mặt hàng StationeryId có thực sự tồn tại và còn kinh doanh không?

Khả năng đáp ứng: stationery.Quantity >= request.Quantity (Kho có đủ hàng giao không?).

Nếu **bất kỳ một** điều kiện nào vi phạm (VD: Thuốc hết hàng,...), phải ném ra Exception và **Rollback toàn bộ** giỏ hàng, tránh tình trạng tạo đơn nửa vời.

13. Hãy đề xuất cách chia Controller, Service và Repository cho chức năng lọc dữ liệu theo Relationship và khoảng giá trị đã làm ở Câu 2.

Controller (StationeriesController): Nhận tham số URL?categoryId=1&minPrice=.... Gọi hàm từ Service. Trả ViewModel ra View.

Service (StationeryService): Gọi Repository để lấy Data Entity. Chứa logic map từ Stationery entity sang class StationeryListItemViewModel, định dạng tiền tệ sang dạng VND.

Repository (StationeryRepository): Chỉ chứa LINQ truy vấn SQL: .Where(...), .Include(s => s.Category), và cực kỳ quan trọng là .AsNoTracking().

14. Hãy mô tả một kịch bản kiểm tra ProductService hoặc Service tương đương bằng Fake Repository theo ba bước Arrange - Act - Assert.

Arrange (Chuẩn bị): Tạo FakeStationeryRepository trả về list tĩnh gồm 2 sản phẩm: Sản phẩm A (Quantity = 2, MinStock = 10), Sản phẩm B (Quantity = 50).

Tạo IOptions<StationeryStoreSettings> giả với ngưỡng mặc định = 5.

Act (Thực thi): Gọi hàm `var result = await _dashboardService.GetDashboardViewModelAsync();`

Assert (Xác nhận): Viết lệnh kiểm tra `Assert.Equal(1, result.NeedReorderCount)` (hi vọng chỉ có Sản phẩm A bị báo sắp hết hàng) và xác nhận `result.TotalStationeries == 2`.

Điều này chứng minh Service tính toán chính xác mà không cần DB thật.